

IOSTREAM_REFERENCE

Core Definition

<iostream> declares the standard stream objects (`cin`, `cout`, `cerr`, `clog` and wide equivalents) and pulls in `<istream>`/`<ostream>`. It is the entry point to the C++ stream hierarchy: formatted and unformatted character I/O layered over stream buffers, with per-stream state flags, formatting flags, and locale.

```
#CPP #IOSTREAM #IO #STREAMS #CIN #COUT #FORMATTED-IO
```

THE STREAM HIERARCHY

```
ios_base          # non-template base: flags, precision, width, locale
├── basic_ios<CharT> # stream state, tie(), rdbuf(), fill()
│   ├── basic_istream<CharT> # >>, get, getline, read, seekg ...
│   │   └── basic_iostream<CharT>
│   └── basic_ostream<CharT> # <<, put, write, seekp, flush ...
│       └── basic_iostream<CharT>
Typedefs:  istream = basic_istream<char>      wistream = basic_istream<wchar_t>
           ostream = basic_ostream<char>      wostream = basic_ostream<wchar_t>
           iostream = basic_iostream<char>    wiostream = basic_iostream<wchar_t>
```

COMPLETE IOSTREAM QUICK REFERENCE

STREAM OBJECTS, MEMBERS & OPERATORS — Target | Operation | Output

```
// =====
// STANDARD STREAM OBJECTS
// =====
std::cin          // istream | Standard input      | Tied to cout, buffered
std::cout         // ostream | Standard output   | Buffered
std::cerr         // ostream | Standard error    | Unit-buffered (unitbuf set)
std::clog         // ostream | Standard log      | Buffered (unlike cerr)
std::wcin, wcout, wcerr, wclog // wide   | Wide-char equivalents | wchar_t streams

// =====
// FORMATTED OUTPUT – operator<<
// =====
os << value       // ostream | Insert formatted value | Returns ostream& (chainable)
                  // Overloads: bool, short, int, long, long long,
                  // unsigned variants, float, double, long double,
                  // const void*, nullptr_t (C++17), streambuf*,
                  // char, const char*, std::string, string_view
os << std::endl    // ostream | Newline + flush      | Returns ostream&
os << std::flush   // ostream | Flush buffer         | Returns ostream&
os << std::ends    // ostream | Insert '\0'           | Returns ostream&

// =====
// UNFORMATTED OUTPUT
// =====
os.put(ch)        // char   | Write single char      | Returns ostream&
os.write(buf, count) // ptr+n | Write n chars raw     | Returns ostream&, no null-term needed
os.flush()        // none  | Flush to device       | Returns ostream&

// =====
```

```

// OUTPUT POSITIONING
// =====
os.tellp()           // none | Get put position | Returns pos_type (-1 on fail)
os.seekp(pos)        // pos_type | Set absolute put pos | Returns ostream&
os.seekp(off, dir)   // off+dir | Seek relative | dir: ios::beg/cur/end

// =====
// FORMATTED INPUT – operator>>
// =====
is >> value          // istream | Extract formatted value | Returns istream&, skips leading ws
// Sets failbit on parse failure; value set to 0 (C++11)
is >> std::ws         // istream | Consume whitespace | Returns istream&
is >> charArray       // char[N] | Extract word | C++20: bounded to N-1; pre-C++20
    UNSAFE

// =====
// UNFORMATTED INPUT – SINGLE CHARACTER
// =====
is.get()             // none | Read one char | Returns int_type (EOF at end)
is.get(ch)           // char& | Read one char into ch | Returns istream&
is.peek()            // none | Look at next char | Returns int_type, does NOT consume
is.unget()           // none | Push back last char | Returns istream&
is.putback(ch)       // char | Push back specific char | Returns istream&

// =====
// UNFORMATTED INPUT – STRINGS & BLOCKS
// =====
is.get(buf, n)        // ptr+n | Read to '\n' (leaves it) | Null-terminates; failbit if 0 chars
is.get(buf, n, delim) // +delim | Read to delim (leaves it) | Null-terminates
is.getline(buf, n)    // ptr+n | Read line, DISCARDS '\n' | Null-terminates; failbit if too long
is.getline(buf, n, delim) // +delim | Read to delim, discards it | Null-terminates
std::getline(is, str) // free fn | Read line into std::string | Returns istream&, grows
    automatically
std::getline(is, str, delim) // +delim | Read to delim into string | Returns istream&
is.read(buf, n)        // ptr+n | Read exactly n chars | Returns istream&; failbit if fewer
is.readsome(buf, n)    // ptr+n | Read available chars | Returns streamsize actually read
is.ignore()            // none | Discard 1 char | Returns istream&
is.ignore(n)           // count | Discard n chars | Returns istream&
is.ignore(n, delim)    // n+delim | Discard to delim incl. | Common:
    ignore(numeric_limits<streamsize>::max(), '\n')
is.gcount()            // none | Chars read by last | Returns streamsize (unformatted ops
    only)

// =====
// INPUT POSITIONING
// =====
is.tellg()           // none | Get get position | Returns pos_type (-1 on fail)
is.seekg(pos)        // pos_type | Set absolute get pos | Returns istream&
is.seekg(off, dir)   // off+dir | Seek relative | dir: ios::beg/cur/end

// =====
// STREAM STATE – CHECKING
// =====
s.good()             // none | No flags set | Returns bool
s.eof()              // none | eofbit set | Returns bool (past-the-end reached)
s.fail()             // none | failbit OR badbit | Returns bool (recoverable error)
s.bad()              // none | badbit set | Returns bool (unrecoverable)
s.operator bool()    // none | !fail() | explicit; enables `while (cin >> x)`
s.operator!()        // none | fail() | Returns bool
s.rdstate()          // none | Get all state flags | Returns iostate bitmask

// =====
// STREAM STATE – MODIFYING
// =====
s.clear()            // none | Reset to goodbit | Returns void
s.clear(state)       // iostate | Set state exactly | Returns void
s.setstate(state)    // iostate | OR flags into state | Returns void
s.exceptions()       // none | Get exception mask | Returns iostate
s.exceptions(mask)   // iostate | Throw on these flags | Throws ios_base::failure

// State flag constants
std::ios::goodbit    // 0 | No errors

```

```

std::ios::eofbit           // bit   | End of file reached
std::ios::failbit         // bit   | Logical/format error
std::ios::badbit          // bit   | Stream integrity lost

// =====
// FORMAT CONTROL – ios_base MEMBERS
// =====
s.flags()                 // none   | Get all format flags   | Returns fmtflags
s.flags(fl)               // fmtflags | Replace all flags     | Returns previous fmtflags
s.setf(fl)                // fmtflags | Set flags (OR)        | Returns previous fmtflags
s.setf(fl, mask)          // fl+mask | Clear mask, set fl    | Returns previous fmtflags
s.unsetf(fl)              // fmtflags | Clear flags           | Returns void
s.precision()             // none   | Get decimal precision | Returns streamsize (default 6)
s.precision(n)            // n       | Set precision         | Returns previous streamsize
s.width()                 // none   | Get field width       | Returns streamsize
s.width(n)                // n       | Set field width       | Returns previous; RESET after next
    output
s.fill()                  // none   | Get fill character    | Returns char_type (default ' ')
s.fill(ch)               // char   | Set fill character    | Returns previous char_type

// =====
// FORMAT FLAGS (ios_base::fmtflags) – use with setf/unsetf
// =====
std::ios::boolalpha       // Print bool as true/false
std::ios::showbase        // Prefix 0x / 0 for hex / octal
std::ios::showpoint       // Always show decimal point
std::ios::showpos         // Show + for positive numbers
std::ios::skipws          // Skip leading whitespace on input (DEFAULT ON)
std::ios::unitbuf         // Flush after each output op
std::ios::uppercase       // Uppercase in hex/scientific (0X1A, 1E5)

// Base field (mask: std::ios::basefield)
std::ios::dec             // Decimal (DEFAULT)
std::ios::oct             // Octal
std::ios::hex             // Hexadecimal

// Float field (mask: std::ios::floatfield)
std::ios::fixed           // Fixed-point notation
std::ios::scientific      // Scientific notation
// fixed|scientific together // Hexfloat (C++11)
// neither set            // Default/general notation

// Adjust field (mask: std::ios::adjustfield)
std::ios::left            // Left-justify, pad right
std::ios::right           // Right-justify, pad left (DEFAULT)
std::ios::internal        // Pad after sign/base prefix

// =====
// SEEK DIRECTION (ios_base::seekdir)
// =====
std::ios::beg             // Offset from beginning
std::ios::cur             // Offset from current position
std::ios::end             // Offset from end

// =====
// BUFFER & STREAM ASSOCIATION
// =====
s.rdbuf()                 // none   | Get stream buffer ptr | Returns basic_streambuf*
s.rdbuf(sb)               // buf*   | Replace buffer         | Returns previous buffer*
s.tie()                   // none   | Get tied ostream      | Returns ostream* (cin tied to cout)
s.tie(os)                 // ostream* | Set tied stream       | Returns previous; tie(nullptr) to untie
os << is.rdbuf()         // buf*   | Dump whole stream      | Idiom: copy file to cout

// =====
// SYNCHRONIZATION WITH C STDIO
// =====
std::ios_base::sync_with_stdio() // none | Query sync state | Returns bool
std::ios_base::sync_with_stdio(false) // bool | Decouple from stdio | Returns previous; BIG speedup

// =====
// LOCALE
// =====

```

```

s.imbue(loc)           // locale | Set stream locale      | Returns previous locale
s.getloc()             // none   | Get stream locale  | Returns locale

// =====
// SENTRY OBJECTS (used when writing custom operators)
// =====
std::istream::sentry sen(is) // istream | Prepare for input    | Skips ws, checks state
std::istream::sentry sen(is,true) // +noskip | Prepare, no ws skip | For unformatted input
std::ostream::sentry sen(os) // ostream | Prepare for output   | Flushes tied stream

// =====
// TYPE ALIASES
// =====
std::streamsize // Signed type for counts/sizes
std::streamoff // Signed type for stream offsets
std::streampos // Type for absolute stream positions (fpos)
std::ios::char_type // char for narrow streams
std::ios::int_type // int for narrow streams (holds char + EOF)
std::ios::pos_type // streampos
std::ios::off_type // streamoff

```

COMMON IOSTREAM PATTERNS & EXAMPLES

Basic I/O

```

#include <iostream>
#include <string>

int main() {
    std::string name;
    int age;

    std::cout << "Name: ";
    std::getline(std::cin, name);

    std::cout << "Age: ";
    std::cin >> age;

    std::cout << name << " is " << age << " years old\n";
    return 0;
}

```

Robust Input Validation Loop

```

#include <iostream>
#include <limits>

int readInt(const char* prompt) {
    int value;
    while (true) {
        std::cout << prompt;
        if (std::cin >> value) {
            // Discard rest of line so trailing junk doesn't leak
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
            return value;
        }
        std::cin.clear(); // Reset failbit FIRST
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        std::cout << "Invalid input, try again.\n";
    }
}

```

The `>>` then `getline` Trap

```
int n;
std::string line;

std::cin >> n;           // Leaves '\n' in the buffer
// std::getline(std::cin, line); // WRONG: reads the leftover empty line

std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
std::getline(std::cin, line); // Correct
```

Reading Until EOF

```
#include <iostream>
#include <string>

int main() {
    std::string word;
    int count = 0;
    while (std::cin >> word) { // Loop ends on EOF or parse failure
        ++count;
    }
    std::cout << "Words: " << count << '\n';

    // Line-based version:
    // std::string line;
    // while (std::getline(std::cin, line)) { /* ... */ }
}
```

Checking Which Failure Occurred

```
int x;
std::cin >> x;

if (std::cin.eof())      std::cerr << "Hit end of input\n";
else if (std::cin.bad()) std::cerr << "Irrecoverable stream error\n";
else if (std::cin.fail()) std::cerr << "Bad format (not a number)\n";
else                    std::cout << "Read " << x << '\n';
```

Exception-Based Error Handling

```
#include <iostream>
#include <fstream>

int main() {
    std::ifstream in;
    in.exceptions(std::ios::failbit | std::ios::badbit);
    try {
        in.open("data.txt");
        int v; in >> v;
    } catch (const std::ios_base::failure& e) {
        std::cerr << "I/O error: " << e.what() << '\n';
    }
}
```

Custom Type Insertion / Extraction

```
#include <iostream>

struct Point {
    int x = 0, y = 0;
```

```
};

std::ostream& operator<<(std::ostream& os, const Point& p) {
    return os << '(' << p.x << ", " << p.y << ')';
}

std::istream& operator>>(std::istream& is, Point& p) {
    char lparen, comma, rparen;
    Point tmp;
    if (is >> lparen >> tmp.x >> comma >> tmp.y >> rparen
        && lparen == '(' && comma == ',' && rparen == ')') {
        p = tmp; // Commit only on success
    } else {
        is.setstate(std::ios::failbit);
    }
    return is;
}
```

Redirecting a Stream

```
#include <iostream>
#include <fstream>

int main() {
    std::ofstream log("out.txt");
    std::streambuf* old = std::cout.rdbuf(); // Save

    std::cout.rdbuf(log.rdbuf()); // Redirect
    std::cout << "This goes to the file\n";

    std::cout.rdbuf(old); // ALWAYS restore
    std::cout << "This goes to the console\n";
}
```

Fast I/O (Competitive Programming)

```
#include <iostream>

int main() {
    std::ios_base::sync_with_stdio(false); // Decouple from C stdio
    std::cin.tie(nullptr); // Don't flush cout before each read

    // Now do NOT mix printf/scanf with cin/cout.
    // Use '\n' instead of std::endl to avoid per-line flushes.
    int n; std::cin >> n;
    std::cout << n << '\n';
}
```

Character-by-Character Processing

```
#include <iostream>
#include <cctype>

int main() {
    char ch;
    int letters = 0, digits = 0, spaces = 0;

    // noskipws so >> does not eat whitespace; or use cin.get(ch)
    while (std::cin.get(ch)) {
        if (std::isalpha(static_cast<unsigned char>(ch))) ++letters;
        else if (std::isdigit(static_cast<unsigned char>(ch))) ++digits;
        else if (std::isspace(static_cast<unsigned char>(ch))) ++spaces;
    }
    std::cout << letters << ' ' << digits << ' ' << spaces << '\n';
}
```

Peeking Ahead

```
if (std::cin.peek() == '-') {
    std::cin.get();    // Consume the sign
    // handle negative case
}
```

IMPORTANT CONCEPTS

Stream State Machine

- `goodbit` (0) means all is well; any other flag stops further extraction.
- `eofbit` alone is **not** an error — it means the end was reached.
- A failed `>>` sets `failbit`; you must `clear()` **before** `ignore()`, since `ignore()` is a no-op on a failed stream.
- `badbit` usually indicates a device-level problem; recovery is not expected.

`eof()` Is Not a Loop Condition

```
while (!std::cin.eof()) { std::cin >> x; /* ... */ } // WRONG: processes last item twice
while (std::cin >> x)   { /* ... */ }               // RIGHT: tests the read itself
```

`eof()` only becomes true *after* a read has already failed.

`width()` Is Sticky-for-One

`precision()` and `fill()` persist across operations. `width()` resets to 0 after the very next formatted output. This is why `setw()` must be re-applied per field.

`endl` VS `'\n'`

`std::endl` writes `'\n'` **and flushes**. In loops that flush is a measurable cost. Use `'\n'` and flush explicitly (or rely on `cerr`, which is unit-buffered) unless you need the data on the device immediately.

Tied Streams

`cin` is tied to `cout` by default: any read on `cin` flushes `cout` first, so prompts appear before input is requested. `cin.tie(nullptr)` breaks this — fast, but prompts may not appear when expected.

Formatted vs Unformatted

- **Formatted** (`<<`, `>>`): applies locale, format flags, skips whitespace, constructs a sentry.
- **Unformatted** (`get`, `getline`, `read`, `write`, `put`): moves characters verbatim; updates `gcount()`.

`get` VS `getline` (C-array versions)

Both stop at the delimiter. `get()` **leaves** the delimiter in the stream (a second call then reads nothing → infinite loop risk); `getline()` **extracts and discards** it. `std::getline(is, std::string&)` is the safer choice — no buffer size to get wrong.

`gcount()` Scope

Only unformatted input operations set `gcount()`. It is reset by the next unformatted op, so read it immediately.

Stream Copying Is Disabled

Streams are non-copyable but movable (C++11). Pass by reference (`std::ostream&`), never by value.

STANDARD STREAM CHEAT SHEET

Stream	Direction	Buffering	Default target	Typical use
<code>cin</code>	input	buffered	<code>stdin</code>	User / piped input
<code>cout</code>	output	buffered	<code>stdout</code>	Normal program output
<code>cerr</code>	output	unit-buffered	<code>stderr</code>	Errors – appears immediately
<code>clog</code>	output	buffered	<code>stderr</code>	Logging – batched, less overhead

BEST PRACTICES

1. **Test the read, not `eof()`** — `while (cin >> x)`, never `while (!cin.eof())`
2. **`clear()` before `ignore()`** — `ignore` does nothing on a failed stream
3. **Prefer `std::getline(is, std::string&)` over C-array `getline`** — no overflow risk
4. **Use `'\n'`, not `endl`**, inside loops
5. **Always restore a swapped `rddbuf()`** — dangling buffers at scope exit are UB
6. **Pass streams by reference** — `void log(std::ostream& os)`
7. **Check state after reads** that matter; don't assume success
8. **Use `sync_with_stdio(false)`** only when not mixing C stdio
9. **Set `boolalpha`** for readable bool output in logs
10. **Save/restore format flags** in library code so callers aren't surprised
11. **Reserve exceptions for genuinely exceptional I/O**; flag checks are cheaper and more idiomatic
12. **Commit-on-success** in custom `operator>>` — don't half-write the target on failure

RELATED HEADERS

```

#include <iostream>    // cin/cout/cerr/clog + <istream> + <ostream>
#include <istream>     // basic_istream, basic_iostream
#include <ostream>     // basic_ostream, endl/flush/ends
#include <iomanip>      // setw, setprecision, quoted ...
#include <fstream>     // file streams
#include <sstream>      // in-memory string streams
#include <streambuf>    // basic_streambuf (custom buffers)
#include <ios>          // ios_base, basic_ios, flags, boolalpha ...
#include <iosfwd>       // forward declarations only (cheap includes)
#include <cstdio>       // C stdio: printf, scanf, FILE*
#include <print>        // C++23: std::print, std::println

```

EXTERNAL RESOURCES

- **Input/Output library:** <https://en.cppreference.com/w/cpp/io>

- **basic_istream**: https://en.cppreference.com/w/cpp/io/basic_istream
 - **basic_ostream**: https://en.cppreference.com/w/cpp/io/basic_ostream
 - **ios_base**: https://en.cppreference.com/w/cpp/io/ios_base
-

Standard: C++11 / C++14 / C++17 / C++20 / C++23 (version noted where relevant) **Last Updated:** September 2026